Project1_VD_Beatriz_Martins

October 13, 2025

1 Project #1

1.1 Data Visualization (LECD, LIACD) 2025/26

Student name: Beatriz Catarina Mateus Martins

1.2 Instructions

- Do not modify the structure of this notebook!
- Fill-in the corresponding cells with the code or text depending on the objective
- You can use this template as a working notebook, or copy&paste your final code here
- It is recommended that you export and submit the rendered version of the notebook
- Submit the notebook not later than the date specified on the Inforestudante webpage of the course

1.3 Data

O dataset escolhido para análise foi o **"Olympics.csv"**, cujos atributos são:

- **ID**: 135 571 linhas, identificador único de cada atleta.
- **Name**: 134 732 nomes diferentes.
- **Sex**: Masculino ou Feminino.
- **Age**: 74 idades diferentes, desde 10 a 97 anos.
- **Height**: 95 valores diferentes, desde 127.0 cm a 226.0 cm.
- **Weight**: 220 valores diferentes, desde 11.3 kg a 97.1 kg.
- **Team**: 1 184 equipas diferentes.
- **NOC**: 230 países diferentes, código de três letras do "National Olympic Committee", ex:. "POR" → Portugal
- **Games**: 51 jogos diferentes, ex.: "1992 Summer".
- **Year**: 35 anos diferentes, desde 1896 até 2016.

- **Season**: Inverno ou verão.
- **City**: 42 cidades diferentes, cidade anfitriã dos jogos.
- **Sport**: 66 modalidades desportivas diferentes.
- **Event**: 765 eventos diferentes, ex.: "Judo Men's Extra-Lightweight".
- **Medal**: Medalha conquistada pelo atleta (ouro, prata ou bronze).

Eliminei os valores nulos da coluna "Medal", considerando apenas os atletas que efetivamente receberam medalha. Além disso, removi 11 linhas duplicadas do dataset. Procedi também à análise da matriz de correlação, com o objetivo de evitar a seleção de atributos fortemente correlacionados, como o peso e a altura, e recorri ao Pairplot para identificar combinações de variáveis potencialmente relevantes para a análise.

Por fim, optei por utilizar as variáveis **"Sex"**, **"Year"** e **"Season"**, agrupando o dataset por ano, estação e sexo, de modo a representar a evolução do número de medalhistas de cada género e em cada estação, ao longo dos anos. Para tal, usei apenas os anos entre 1924 e 1992 que incluem ambas as estações de competição, conforme demonstrado na fase de pré-processamento.

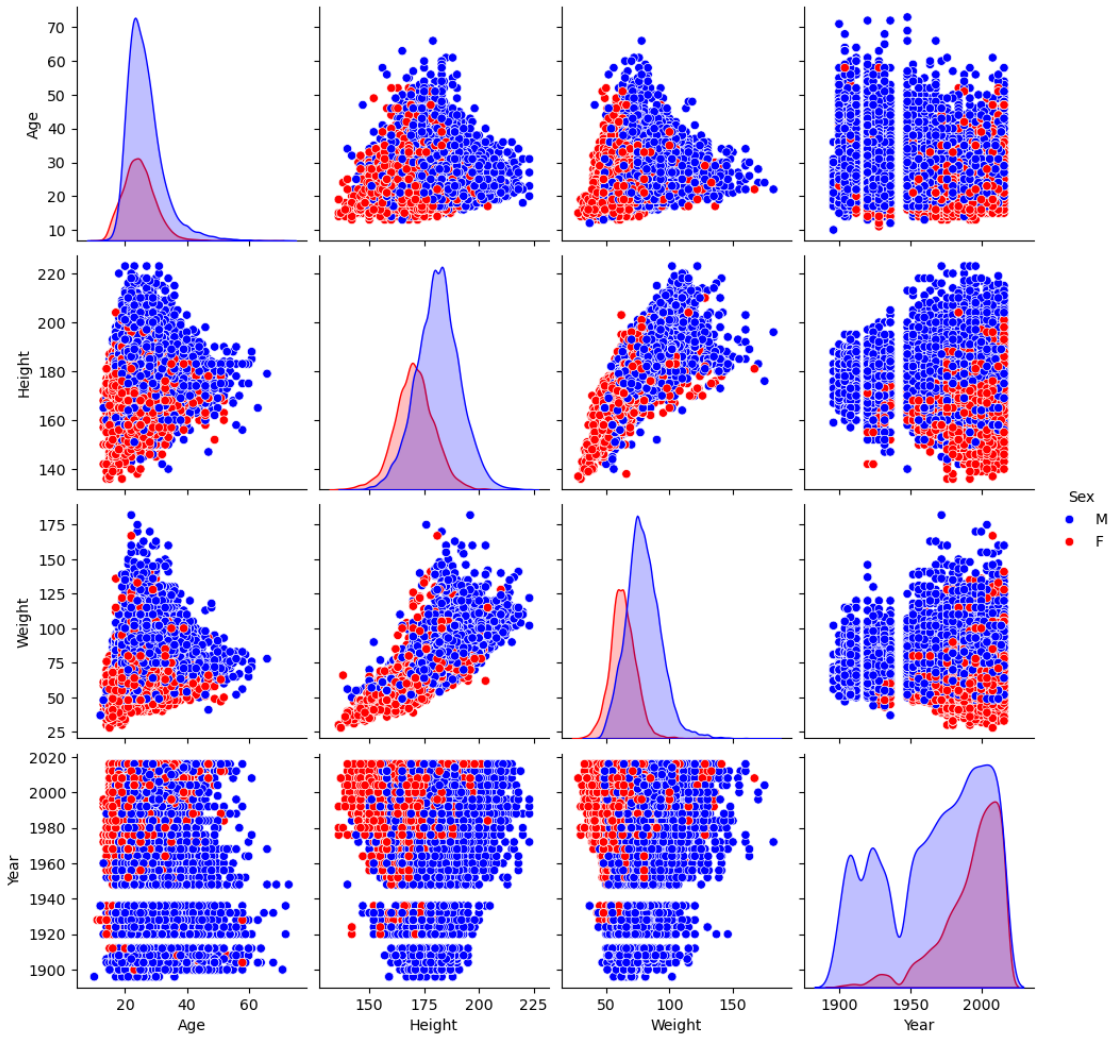
[3]:
```
import pandas as pd
import numpy as np
import plotly.graph_objects as go
import seaborn as sns
import matplotlib.pyplot as plt

# Descarregar e analisar o datset
df = pd.read_csv('olympics.csv')
print(f"(linhas, colunas) = {df.shape}","\n")
print(df.info(), "\n")
print(df.head(),"\n")
for i, col in enumerate(df.columns):
    if i in [3, 4, 5, 9]: # Colunas numéricas
        print(f"Número de valores diferentes em {col}: {df[col].nunique()},␣
 ↪(desde {df[col].min()} a {df[col].max()})")
    else:
        print(f"Número de valores diferentes em {col}: {df[col].nunique()}") #␣
 ↪nº de valores diferentes na coluna

# Valores nulos
missing_values = df.isnull().sum()
print("\nValores nulls por coluna:")
print(missing_values, "\n")

# Retirar as pessoas que não receberam medalha
df = df.dropna(subset=['Medal'])
```

```python
# linhas duplicadas
duplicate_rows = df.duplicated().sum()
df = df.drop_duplicates()
print(f'Linhas duplicadas: {duplicate_rows}', "\n")

# Matriz de correlações
numeric_df = df.select_dtypes(include=['float64', 'int64'])  # selecionar só as␣
↪variáveis numéricas
corr = numeric_df.corr()
fig = go.Figure(data=go.Heatmap(z=corr.values, # valores das correlações
                                x=corr.columns,
                                y=corr.columns,
                                colorscale="RdBu", # red: -1, blue: 1
                                zmin=-1, zmax=1))  # correlações variam entre␣
↪-1 e 1
fig.update_layout(title="Mapa de Correlações entre Atributos")
fig.show()

# Pairplot
cols_to_use = ["Age", "Height", "Season", "Weight", "Year", "Sex"]  #␣
↪experimetar com várias variáveis
sns.pairplot(df[cols_to_use], hue = "Sex", diag_kind = "kde", palette = {"M":␣
↪"blue", "F": "red"}) # mostra a densidade na diagonal
plt.show()

# Selecionar os três atributos
cols = ['Sex', 'Year', 'Season']
df = df[cols]

# Descobrir em que anos houve duas estações com jogos
season_count_per_year = df.groupby('Year')['Season'].nunique()  # matriz com nº␣
↪de seasons de cada ano
for ano, n_estacoes in season_count_per_year.items():
    print(f"Ano {ano}: {n_estacoes} {"estação" if n_estacoes == 1 else␣
↪"estações"} com provas")

# Lista dos anos que têm duas seasons de provas
anos_validos = [1924, 1928, 1932, 1936, 1948, 1952, 1956, 1960, 1964, 1968,␣
↪1972, 1976, 1980, 1984, 1988, 1992]

# Filtrar apenas as linhas correspondentes a esses anos
df = df[df['Year'].isin(anos_validos)]

# Renomear as colunas para algo mais legível
df['Sex'] = df['Sex'].replace({'M': 'Homens', 'F': 'Mulheres'})
```

```python
# Conta quantos registos há por ano, estação e sexo (coloca 0 onde não há)
final_df = df.groupby(['Year', 'Season', 'Sex']).size().unstack(fill_value=0)

print("\nDATASET FINAL:")
print(final_df)
print(df.columns)
```

```
(linhas, colunas) = (271116, 15)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 271116 entries, 0 to 271115
Data columns (total 15 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   ID      271116 non-null  int64
 1   Name    271116 non-null  object
 2   Sex     271116 non-null  object
 3   Age     261642 non-null  float64
 4   Height  210945 non-null  float64
 5   Weight  208241 non-null  float64
 6   Team    271116 non-null  object
 7   NOC     271116 non-null  object
 8   Games   271116 non-null  object
 9   Year    271116 non-null  int64
 10  Season  271116 non-null  object
 11  City    271116 non-null  object
 12  Sport   271116 non-null  object
 13  Event   271116 non-null  object
 14  Medal   39783 non-null   object
dtypes: float64(3), int64(2), object(10)
memory usage: 31.0+ MB
None

   ID                      Name Sex   Age  Height  Weight            Team  \
0   1                 A Dijiang   M  24.0   180.0    80.0           China
1   2                  A Lamusi   M  23.0   170.0    60.0           China
2   3       Gunnar Nielsen Aaby   M  24.0     NaN     NaN         Denmark
3   4      Edgar Lindenau Aabye   M  34.0     NaN     NaN  Denmark/Sweden
4   5  Christine Jacoba Aaftink   F  21.0   185.0    82.0     Netherlands

   NOC        Games  Year  Season       City          Sport  \
0  CHN  1992 Summer  1992  Summer  Barcelona     Basketball
1  CHN  2012 Summer  2012  Summer     London           Judo
2  DEN  1920 Summer  1920  Summer  Antwerpen       Football
3  DEN  1900 Summer  1900  Summer      Paris     Tug-Of-War
4  NED  1988 Winter  1988  Winter    Calgary  Speed Skating

                                Event Medal
```

```
0       Basketball Men's Basketball   NaN
1      Judo Men's Extra-Lightweight   NaN
2           Football Men's Football   NaN
3       Tug-Of-War Men's Tug-Of-War  Gold
4  Speed Skating Women's 500 metres   NaN

Número de valores diferentes em ID: 135571
Número de valores diferentes em Name: 134732
Número de valores diferentes em Sex: 2
Número de valores diferentes em Age: 74, (desde 10.0 a 97.0)
Número de valores diferentes em Height: 95, (desde 127.0 a 226.0)
Número de valores diferentes em Weight: 220, (desde 25.0 a 214.0)
Número de valores diferentes em Team: 1184
Número de valores diferentes em NOC: 230
Número de valores diferentes em Games: 51
Número de valores diferentes em Year: 35, (desde 1896 a 2016)
Número de valores diferentes em Season: 2
Número de valores diferentes em City: 42
Número de valores diferentes em Sport: 66
Número de valores diferentes em Event: 765
Número de valores diferentes em Medal: 3

Valores nulls por coluna:
ID             0
Name           0
Sex            0
Age         9474
Height     60171
Weight     62875
Team           0
NOC            0
Games          0
Year           0
Season         0
City           0
Sport          0
Event          0
Medal     231333
dtype: int64

Linhas duplicadas: 11
```




```
Ano 1896: 1 estação com provas
Ano 1900: 1 estação com provas
Ano 1904: 1 estação com provas
Ano 1906: 1 estação com provas
Ano 1908: 1 estação com provas
Ano 1912: 1 estação com provas
Ano 1920: 1 estação com provas
Ano 1924: 2 estações com provas
Ano 1928: 2 estações com provas
Ano 1932: 2 estações com provas
Ano 1936: 2 estações com provas
Ano 1948: 2 estações com provas
Ano 1952: 2 estações com provas
Ano 1956: 2 estações com provas
Ano 1960: 2 estações com provas
```

```
Ano 1964: 2 estações com provas
Ano 1968: 2 estações com provas
Ano 1972: 2 estações com provas
Ano 1976: 2 estações com provas
Ano 1980: 2 estações com provas
Ano 1984: 2 estações com provas
Ano 1988: 2 estações com provas
Ano 1992: 2 estações com provas
Ano 1994: 1 estação com provas
Ano 1996: 1 estação com provas
Ano 1998: 1 estação com provas
Ano 2000: 1 estação com provas
Ano 2002: 1 estação com provas
Ano 2004: 1 estação com provas
Ano 2006: 1 estação com provas
Ano 2008: 1 estação com provas
Ano 2010: 1 estação com provas
Ano 2012: 1 estação com provas
Ano 2014: 1 estação com provas
Ano 2016: 1 estação com provas

DATASET FINAL:
Sex          Homens  Mulheres
Year Season
1924 Summer     786        46
     Winter     124         6
1928 Summer     637        97
     Winter      83         6
1932 Summer     585        62
     Winter      86         6
1936 Summer     829        88
     Winter      99         9
1948 Summer     751       101
     Winter     120        15
1952 Summer     757       140
     Winter     118        18
1956 Summer     749       144
     Winter     123        27
1960 Summer     755       156
     Winter     108        39
1964 Summer     829       200
     Winter     140        46
1968 Summer     842       215
     Winter     153        46
1972 Summer     965       250
     Winter     154        45
1976 Summer     931       389
     Winter     160        51
```

```
1980 Summer     950       434
     Winter     167        51
1984 Summer     982       494
     Winter     168        54
1988 Summer    1028       554
     Winter     200        63
1992 Summer    1120       592
     Winter     219        99
Index(['Sex', 'Year', 'Season'], dtype='object')
```

1.4 Visualization model #1

- **Nota:** Foi necessário reagrupar novamente os dados para aceder corretamente às colunas. O dataset não foi modificado neste processo, foi apenas dividido em dois dataframes — um para o Inverno e outro para o Verão — para facilitar a sua manipulação.

[6]:
```
# Separar as amostras por estação (verão e inverno), que ainda não corresponde␣
↪ao Datset Final
summer = df[df['Season'] == 'Summer']
winter = df[df['Season'] == 'Winter']
print(summer)
print(winter)

# Agrupar por ano e sexo para reconstruir o dataset final (sem alterar os dados)
summer_counts = summer.groupby(['Year','Sex']).size().unstack(fill_value=0) #␣
↪Dados do verão
winter_counts = winter.groupby(['Year','Sex']).size().unstack(fill_value=0) #␣
↪Dados do inverno
print(summer_counts)
print(winter_counts)


fig1 = go.Figure()

# ---- Verão ----
fig1.add_trace(go.Scatter(
    x = summer_counts.index,
    y = summer_counts['Homens'],
    mode = 'lines+markers',
    name = 'Verão (Homens)',
    line = dict(color = '#2196f3'),
    marker = dict(size = 10, symbol = 'circle')
))
fig1.add_trace(go.Scatter(
    x = summer_counts.index,
    y = summer_counts['Mulheres'],
    mode = 'lines+markers',
    name = 'Verão (Mulheres)',
```

```
    line = dict(color = '#f74780'),
    marker = dict(size = 10, symbol = 'circle')
))

# ---- Inverno ----
fig1.add_trace(go.Scatter(
    x = winter_counts.index,
    y = winter_counts['Homens'],
    mode = 'lines+markers',
    name = 'Inverno (Homens)',
    line = dict(color = '#c3e3fd'),
    marker = dict(size = 10, symbol = 'square')
))
fig1.add_trace(go.Scatter(
    x = winter_counts.index,
    y = winter_counts['Mulheres'],
    mode = 'lines+markers',
    name = 'Inverno (Mulheres)',
    line = dict(color = '#ffc1d5'),
    marker = dict(size = 10, symbol = 'square')
))

fig1.update_layout(
    title = dict(
        text = '<b>Evolução dos medalhistas por sexo e estação ao longo dos␣
 ↪anos</b>',
        font = dict(color = '#1E88E5', size = 27, family = "Calibri"),
        x = 0.4
    ),
    xaxis = dict(
        title = dict(text = 'Ano', font = dict(size = 16, color = "#020202")),
        gridcolor = '#FFB5C0',
        tickvals = summer_counts.index, # aparecer só os anos que houve provas␣
 ↪(eixo x)
        tickangle = -40 # valores do eixo x inclinados
    ),
    yaxis = dict(
        title = dict(text = 'Número de Atletas', font = dict(size = 16, color =␣
 ↪"#020202")),
        gridcolor = '#FFB5C0',
        rangemode = 'tozero' # eixo do y começa em zero
    ),
    legend_title = dict(text = 'Categorias:', font = dict(size = 14)),
    plot_bgcolor = "#FFF9FA"  # cor de fundo do gráfico
)

fig1.show()
```

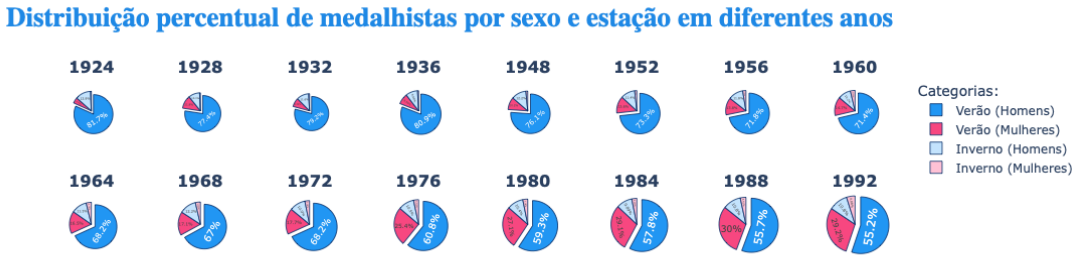
```
             Sex  Year  Season
41        Homens  1948  Summer
42        Homens  1948  Summer
44        Homens  1948  Summer
48        Homens  1948  Summer
50        Homens  1952  Summer
…          …   …     …
271048    Homens  1992  Summer
271076  Mulheres  1952  Summer
271078  Mulheres  1956  Summer
271080  Mulheres  1964  Summer
271082    Homens  1980  Summer

[17458 rows x 3 columns]
             Sex  Year  Season
60        Homens  1992  Winter
61        Homens  1992  Winter
110       Homens  1952  Winter
113       Homens  1960  Winter
672       Homens  1924  Winter
…          …   …     …
270480  Mulheres  1980  Winter
270641    Homens  1992  Winter
270827    Homens  1988  Winter
270876    Homens  1988  Winter
270878    Homens  1988  Winter

[2803 rows x 3 columns]
Sex   Homens  Mulheres
Year
1924     786        46
1928     637        97
1932     585        62
1936     829        88
1948     751       101
1952     757       140
1956     749       144
1960     755       156
1964     829       200
1968     842       215
1972     965       250
1976     931       389
1980     950       434
1984     982       494
1988    1028       554
1992    1120       592
Sex   Homens  Mulheres
Year
```

```
1924     124         6
1928      83         6
1932      86         6
1936      99         9
1948     120        15
1952     118        18
1956     123        27
1960     108        39
1964     140        46
1968     153        46
1972     154        45
1976     160        51
1980     167        51
1984     168        54
1988     200        63
1992     219        99
```

1.5 Visualization model #2

[8]:
```
from plotly.subplots import make_subplots
import math

# Lista de anos disponíveis no dataset de inverno
anos = winter_counts.index.tolist()

# Número de colunas e linhas dos subplots
n_cols = 8
n_rows = math.ceil(len(anos) / n_cols)

# Criar subplots
fig2 = make_subplots(
    rows = n_rows,
    cols = n_cols,
    # define cada célula como um gráfico circular (domain)
    specs = [[{'type':'domain'}] * n_cols for i in range(n_rows)],
    # título de cada subplot é o ano
    subplot_titles=[f"<span style='font-weight:bold; color:'rgb(8,48,107)';␣
↪font-size:15.5px;'>{ano}</span>" for ano in anos]
)

# Adicionar um gráfico Pie para cada ano
for i, ano in enumerate(anos):

    fig2.add_trace(
        go.Pie(
            labels = ['Verão (Homens)', 'Verão (Mulheres)', 'Inverno (Homens)',␣
↪'Inverno (Mulheres)'],
```

```
            values = [
                summer_counts.loc[ano, 'Homens'],
                summer_counts.loc[ano, 'Mulheres'],
                winter_counts.loc[ano, 'Homens'],
                winter_counts.loc[ano, 'Mulheres']
            ],
            marker_colors = ['#2196f3', '#f74780', '#c3e3fd', '#ffc1d5'],
            hole = 0.0,   # tamanho do buraco central
            showlegend = (i==0),  # mostrar legenda apenas uma vez para o␣
 ↪primeiro Pie
            textinfo = 'percent',  # mostrar a percentagem
            insidetextfont = dict(size = 12), # tamanho da percentagem
            textposition = 'inside',  # percentagem dentro da fatia
            sort = False,    # manter sempre a mesma ordem para vizualizar␣
 ↪melhor
            direction = 'clockwise',  # sentido dos ponteiros
            rotation = 0,   # a primeira fatia começa sempre no mesmo angulo do␣
 ↪círculo
            pull = [0.2, 0, 0, 0], # destacar a categoria "Verão (homens)" para␣
 ↪visualizar melhor
            scalegroup = 'one' # pies maiores para anos com maior nº total de␣
 ↪medalhistas
        ),
        row =  i // n_cols + 1, # divisão inteira
        col = i % n_cols + 1    # resto da divisão
    )

fig2.update_traces(marker=dict(line=dict(color='rgb(8,48,107)', width=0.8))) #␣
 ↪fatias destacadas a preto

fig2.update_layout(
    title = dict(
        text = '<b>Distribuição percentual de medalhistas por sexo e estação em␣
 ↪diferentes anos</b>',
        font = dict(color = '#1E88E5', size = 27, family = "Calibri"),
        x = 0.43
    ),
    legend_title = dict(text = 'Categorias:', font = dict(size = 14))
)

fig2.show()
```



1.6 Visualization model #3

[10]:
```
fig3 = go.Figure(data = [
    go.Bar(x = winter_counts.index, y = summer_counts['Homens'], name = 'Verão␣
↪(Homens)', marker_color = '#2196f3', offsetgroup = "Summer"),
    go.Bar(x = winter_counts.index, y = summer_counts['Mulheres'], name =␣
↪'Verão (Mulheres)', marker_color = '#f74780', offsetgroup = "Summer"),
    go.Bar(x = winter_counts.index, y = winter_counts['Homens'], name =␣
↪'Inverno (Homens)', marker_color = '#c3e3fd', offsetgroup = "winter"),
    go.Bar(x = winter_counts.index, y = winter_counts['Mulheres'], name =␣
↪'Inverno (Mulheres)', marker_color = '#ffc1d5', offsetgroup = "winter")

])

fig3.update_traces(marker_line_color='rgb(8,48,107)', marker_line_width=0.8) #␣
↪linhas pretas à volta das barras

fig3.update_layout(
    barmode = 'stack',  # barras empilhados em cada ano para cada grupo (summer␣
↪e winter)
    title = dict(
        text = '<b>Comparação anual de medalhistas por estação e sexo</b>',
        font = dict(color = '#1E88E5', size = 27, family = "Calibri")
        #x = 0.4
    ),
    xaxis = dict(
        title = dict(text = 'Ano', font = dict(size = 16, color = "#020202")),
        gridcolor = '#FFB5C0',
        tickvals = winter_counts.index,
        tickangle = -40  # rodar ligeiramente os anos para se perceberem melhor
    ),
    yaxis = dict(
```

```
        title = dict(text = 'Número de Medalhistas', font = dict(size = 16,␣
↪color = "#020202")),
        gridcolor = '#FFB5C0'
    ),
    legend_title = dict(text = 'Categorias:', font = dict(size = 14)),
    plot_bgcolor = "#FFF9FA"  # cor de fundo do gráfico #fdf1f1
)

fig3.show()
```

1.7 Visualization model #4

[12]:
```
# fazer uma cópia para não afetar as células anteriores
winter_copy = winter_counts.copy()
summer_copy = summer_counts.copy()

# Acrescentar os anos sem provas para visualizar esse intervalo sem jogos
summer_copy.loc[1940] = [0, 0]
summer_copy.loc[1944] = [0, 0]
winter_copy.loc[1940] = [0, 0]
winter_copy.loc[1944] = [0, 0]

# Reordenar os índices por ano
summer_copy = summer_copy.sort_index()
winter_copy = winter_copy.sort_index()


fig4 = go.Figure()

# ---- Verão ----
fig4.add_trace(go.Barpolar(
    r = summer_copy['Homens'], # eixo do raio (nº de medalhistas)
    theta = winter_copy.index.astype(str), # eixo circular (anos)
```

```
    name = 'Verão (Homens)',
    marker = dict(
        color = '#2196f3',
        line = dict(color = 'rgb(8,48,107)', width = 0.5) # linha à volta das␣
 ↪fatias
    )
))

fig4.add_trace(go.Barpolar(
    r = summer_copy['Mulheres'],
    theta = winter_copy.index.astype(str),
    name = 'Verão (Mulheres)',
    marker = dict(
        color = '#f74780',
        line = dict(color = 'rgb(8,48,107)', width = 0.5)
    )
))

# ---- Inverno ----
fig4.add_trace(go.Barpolar(
    r = winter_copy['Homens'],
    theta = winter_copy.index.astype(str),
    name = 'Inverno (Homens)',
    marker = dict(
        color = '#c3e3fd',
        line = dict(color = 'rgb(8,48,107)', width = 0.5)
    )
))

fig4.add_trace(go.Barpolar(
    r = winter_copy['Mulheres'],
    theta = winter_copy.index.astype(str),
    name = 'Inverno (Mulheres)',
    marker = dict(
        color = '#ffc1d5',
        line = dict(color = 'rgb(8,48,107)', width = 0.5)
    )
))

fig4.update_layout(
    title = dict(text = '<b>Crescimento global do número de medalhistas ao␣
 ↪longo dos anos</b>',
                font = dict(color = '#1E88E5', size = 27, family = "Calibri"),
                x = 0.4
    ),
    legend_title = dict(text = 'Categorias:', font = dict(size = 14)),
    polar = dict(
```

```
        # eixo no raio do círculo
        radialaxis = dict(
            title = '<b>Nº DE MEDALHISTAS<b>',
            color = "#000080",
            linecolor = "#000080",
            tickfont = dict(color = "#000080", size = 10, family = "Arial␣
↪Black"), gridcolor = '#FFB5C0'),
        # datas na eixo circular
        angularaxis = dict(
            direction = 'clockwise',
            # lista das posições angulares
            tickvals = [1924, 1928, 1932, 1936, 0, 0, 1948, 1952, 1956, 1960,␣
↪1964, 1968, 1972, 1976, 1980, 1984, 1988, 1992], # 1940, 1944 não têm provas
            ticktext = [
                    f"<span style='font-weight:bold; font-size:{size}px; color:
↪#000080'>{year}</span>" # valores a negrito e com tamaho decrescente
                    for size, year in zip([20, 18, 17.7, 17.2, 16.8, 16.2, 15.
↪6, 15, 14.5, 14, 13.5, 13, 12.5, 12.5, 12, 11.7, 11.5, 11.5], winter_copy.
↪index)
                ],
            gridcolor = '#FFB5C0'
        ),
        bgcolor = "#FFF9FA" # cor do fundo do gráfico polar #fdf1f1
))

fig4.show()
```

1.8 Critical reflection

As quatro visualizações criadas permitiram compreender a evolução do número de medalhistas por sexo e estação ao longo dos anos. Notou-se que há muito menos medalhistas nos Jogos de Inverno, provavelmente devido ao menor número de modalidades nessa estação. **O primeiro gráfico**, um scatter plot com as quatro categorias separadas, mostrou a evolução individual de cada grupo, permitindo identificar, por exemplo, que o número de homens no inverno superou o das mulheres no verão em 1924, 1932, 1936 e 1948. Por representar cada categoria isoladamente, é mais adequado à exploração do que à comunicação global.

O segundo gráfico, composto por vários pie charts de diferentes anos, evidenciou a predominância masculina no verão e a reduzida presença feminina no inverno. Devido ao parâmetro scalegroup, é também possível observar o aumento do número total de medalhistas, já que os pie charts crescem até ao ano mais recente. **O terceiro gráfico**, um bar plot com as duas estações lado a lado e os géneros empilhados, tornou-se o mais intuitivo e comunicativo, pois permite observar e comparar a evolução tanto do número de medalhistas em cada estação (menos medalhistas no inverno), como a sua partição por sexo ao longo dos anos (maior presença dos homens). Além disso, revela de forma visualmente clara o intervalo de anos sem provas (1936 a 1948), associado ao período da Segunda Guerra Mundial (1939 a 1945), sendo por isso uma visualização particularmente memorável. Em alternativa, a Visualização Extra, sem agrupamento por estação, permite identificar a categoria mais presente em todos os anos (homens no verão).

Por fim, **o quarto gráfico**, um barpolar chart, representa todos os anos num único círculo, com cada fatia dividida pelas quatro categorias. Este formato comunica, de forma global e visualmente apelativa, o crescimento contínuo do número total de medalhistas, atingindo o ponto máximo no ano mais recente. Assim como na visualização anterior, também é visível o intervalo de anos em que não houve provas. No entanto, devido à disposição circular, as categorias menores tendem a parecer ainda mais reduzidas junto ao centro, enquanto que as maiores se destacam mais na periferia. Para mitigar este efeito, posicionei o verão no centro e o inverno na periferia. Assim, este gráfico é eficaz, mas pode distorcer a perceção das proporções de cada fatia.

1.9 References

- https://datavizproject.com/
- https://datavizcatalogue.com/
- https://www.data-to-viz.com/
- https://plotly.com/python/creating-and-updating-figures/
- https://plotly.com/python/graph-objects/
- https://plotly.com/python/line-and-scatter/
- https://plotly.com/python/pie-charts/
- https://plotly.com/python/bar-charts/
- https://plotly.com/python/polar-chart/
- https://plotly.com/python/reference/layout/polar/

1.10 Visualização Extra

[16]:
```
fig5 = go.Figure(data = [
    go.Bar(x = winter_counts.index, y = summer_counts['Homens'], name = 'Verão␣
↪(Homens)', marker_color = '#2196f3'),
    go.Bar(x = winter_counts.index, y = summer_counts['Mulheres'], name =␣
↪'Verão (Mulheres)', marker_color = '#f74780'),
    go.Bar(x = winter_counts.index, y = winter_counts['Homens'], name =␣
↪'Inverno (Homens)', marker_color = '#c3e3fd'),
    go.Bar(x = winter_counts.index, y = winter_counts['Mulheres'], name =␣
↪'Inverno (Mulheres)', marker_color = '#ffc1d5')

])

fig5.update_traces(marker_line_color='rgb(8,48,107)', marker_line_width=0.3) #␣
↪linhas pretas à volta das barras
```

```
fig5.update_layout(
    barmode = 'stack',  # barras (dos homens e mulheres de cada estação)␣
 ↪empilhados em cada ano
    title = dict(
        text = '<b>Distribuição Total de Medalhistas por Sexo e Estações do Ano␣
 ↪desde 1924 até 1992</b>',
        font = dict(color = '#1E88E5', size = 27, family = "Calibri")
        #x = 0.4
    ),
    xaxis = dict(
        title = dict(text = 'Ano', font = dict(size = 16, color = "#020202")),
        gridcolor = '#FFB5C0',
        tickvals = [1924, 1928, 1932, 1936, 1948, 1952, 1956, 1960, 1964, 1968,␣
 ↪1972, 1976, 1980, 1984, 1988, 1992], # aparecer os anos 1940 e 1944 que não␣
 ↪houve provas
        tickangle = -40  # rodar ligeiramente os anos para se perceberem melhor
    ),
    yaxis = dict(
        title = dict(text = 'Número de Medalhistas', font = dict(size = 16,␣
 ↪color = "#020202")),
        gridcolor = '#FFB5C0'
    ),
    legend_title = dict(text = 'Categorias:', font = dict(size = 14)),
    plot_bgcolor = "#FFF9FA"  # cor de fundo do gráfico #fdf1f1
)

fig5.show()
```

[ ]:

```

```